

COMPASS: Towards Cost-Optimal MPC-as-a-Service for Secure Analytical Queries

Shimin Luo, Donghyun Sohn, Jennie Rogers

{shiminluo2029, donghyun.sohn}@u.northwestern.edu, jennie@northwestern.edu

Northwestern University



NORTHWESTERN
UNIVERSITY

Background

Secure Query Evaluation:

- A technique for executing queries using cryptographic protocols or trusted hardware to ensure full obliviousness.
- Full obliviousness hides data access patterns, communication patterns, and instruction traces to prevent any leakage of private inputs.

Secure Multiparty Computation (MPC):

- Queries are translated into Data Independence gate circuits.
- Circuit evaluation follows a fixed topological order, ensuring identical instruction traces regardless of private inputs.
- Parties interact to evaluate gates, with AND gates driving the primary communication overhead.

Cloud Infrastructure:

- AWS EC2 instances: Offers five main families of instances optimized for specific resource.
- Cloud-Native Database (DBMS)
 - Automated Management: Orchestrates resources transparently, abstracting hardware complexities from the user.
 - Scalability & Elasticity: Leverages storage-compute disaggregation for independent, workload-driven scaling.
 - Serverless Efficiency: Pay-as-you-go, aligns costs with real-time usage.
- Untrusted third-party environment; Data is encrypted as secret shares during execution.

Motivation

Selecting the Cost-Optimal cloud resources for secure queries is non-trivial !

- Privacy-Performance Trade-off:** SQL-over-MPC engines run OOMs slower than plaintext execution due to cryptographic overhead.
 - Extended Latency Amplifies Cost!
 - Bottleneck: Disk I/O -> CPU-intensive cryptography and network communication.
- Complexity of Manual Configuration:**
 - Selecting from 700+ AWS instances is daunting. Users must manually navigate unpredictable cost-performance trade-offs across diverse hardware.
 - The highest-cost instance does not guarantee to yield the best performance for specific MPC workloads. Too many instances to try them all!
- Absence of Automated Management**
 - There is no MPC-as-a-Service (MaaS) frameworks capable of abstracting infrastructure management to bridge the usability gap between secure analytics and modern serverless services (Lambda & Cloud-Native DBs).
- Lack of Cost Efficiency Strategy:**
 - Suboptimal User Choices
 - Inefficient Resource Utilization

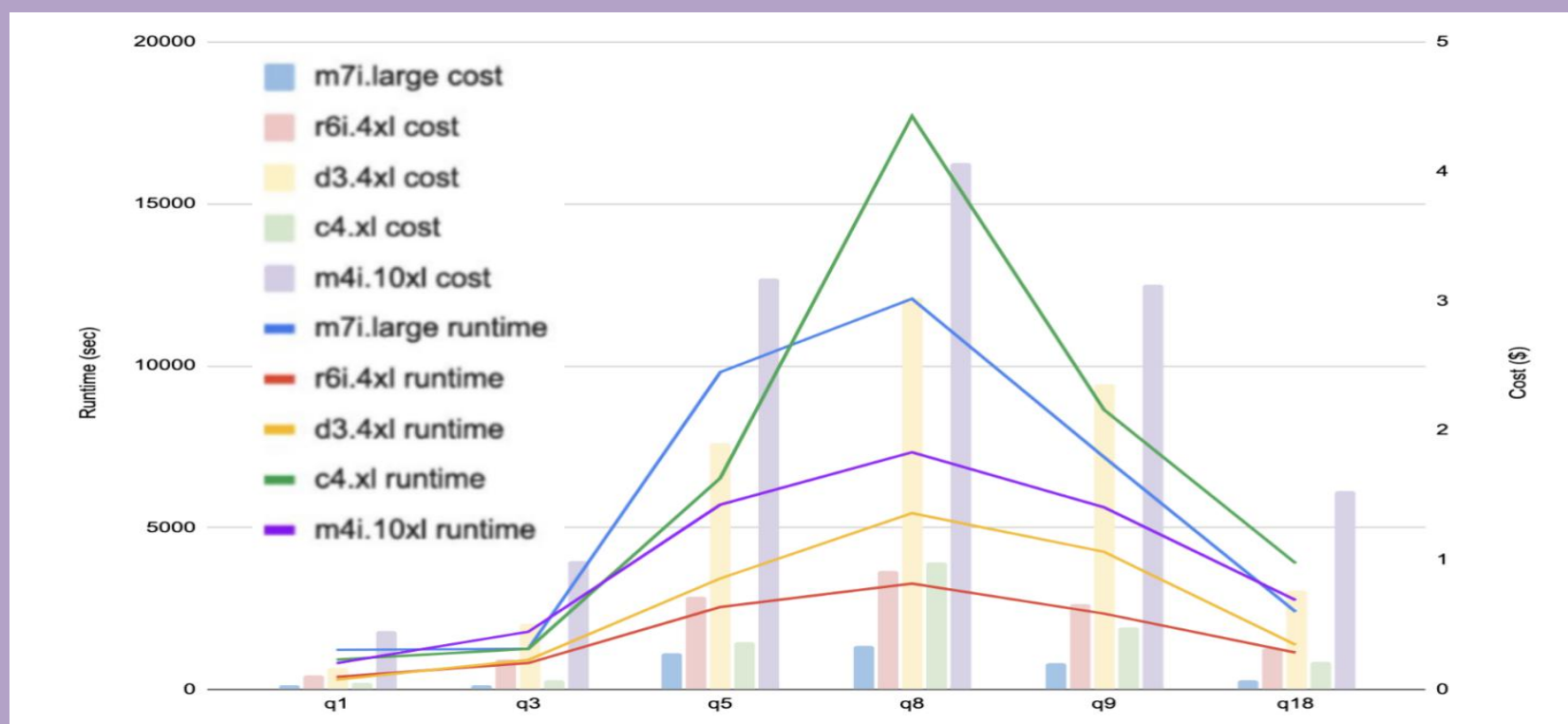


Fig. Runtimes and costs for TPC-H queries on 5 AWS EC2 instances under different optimizations

Demand	Strong privacy (obliviousness) + Good performance + Low cost
Current Reality	Manual configuration + Inefficient resource use + Unpredictable pricing
COMPASS	Automated Management + QPP-driven deployment = Cost-Optimal MaaS

Challenges:

- No existing methodology effectively guides secure query engines toward cost-optimal deployments in the cloud.
- MPC workloads consist of diverse, circuit-heavy operators (joins, filters, aggregates) whose performance scales non-linearly with hardware specifications.
- Core difficulty lies in extrapolating performance across 700+ instances (under varied parallelism) based on a limited set of sampled benchmarks.

Experimental Setup & Preliminary/Expected Result

- Secure Engine: VaultDB + EMP-toolkit (Semi-honest 2PC).
- Workloads: TPC-H SF 0.01 to 0.1; Infrastructure: AWS EC2 families.

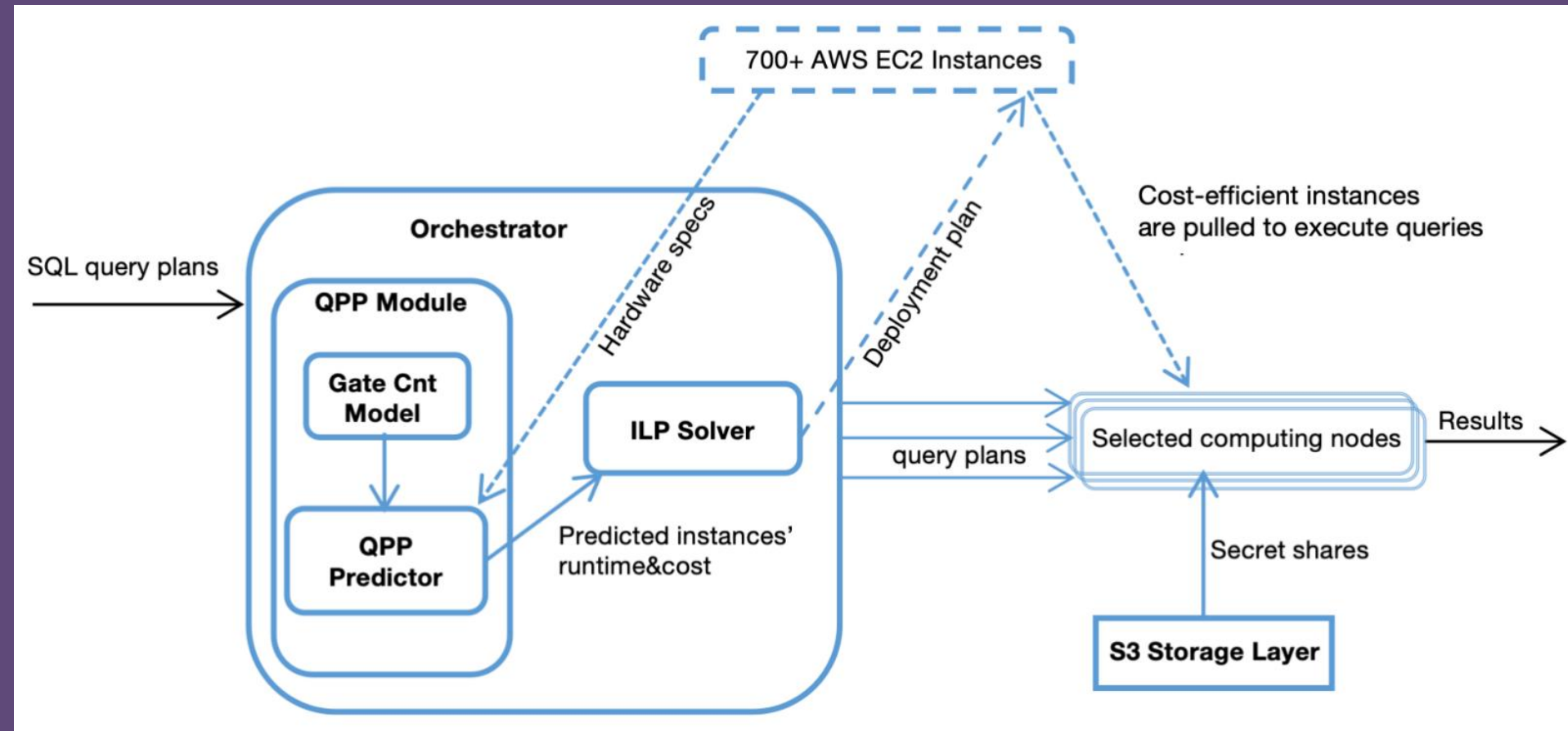
- High Correlation: Predicted vs. actual runtime achieves RSQ > 0.99; accuracy > 85%.
- Robust Generalization: Average error < 20% for unseen instances, aiming to achieve zero-shot predictive power.
- Cost-Efficiency: The Deployment Selector can prevent suboptimal choices & naive baselines (e.g., picking the cheapest or largest instance).
- Versatility: Consistent accuracy across diverse query workloads, ensuring reliability for complex analytical tasks.

Future Work

- Adaptive Caching & Learning
 - Cache: Store historical operator runtimes to enable instant forecasting.
 - Periodically retrain the QPP model with new data to improve accuracy.
- Extend support beyond AWS to GCP, Azure, and hybrid environments.
- Asymmetric Resource Pairing for SH-2PC:
 - The Garbler (P1) handles much higher AES load than the Evaluator (P2).
 - CPU usage (same instance): P1 > 90%, P2: 30%–40%.

Research Problem & COMPASS Architecture

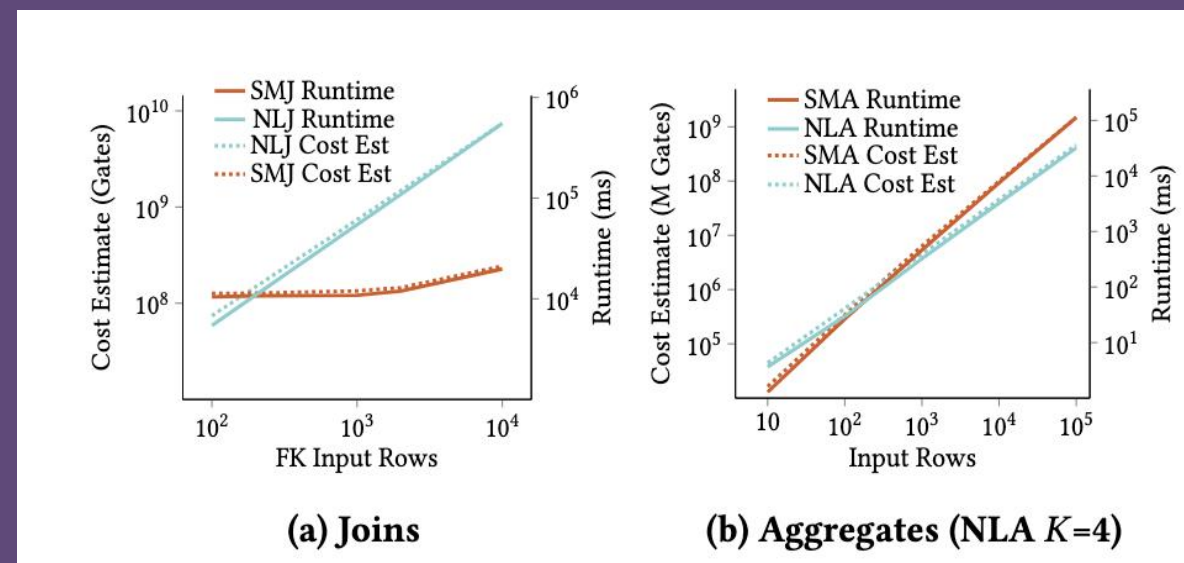
Given a set of analytical queries, COMPASS leverages a Query Performance Prediction (QPP) module and an Integer Linear Programming (ILP) solver to identify a **cost-efficient deployment plan**. It **balances high-performance execution with strong security guarantees**, delivering results within a **MPC-as-a-Service (MaaS) framework**.



- Input: Secure SQL query plans generated by the oblivious query optimizer.
- Gate Count Model: Estimate the total number of AND gates given the query plan and input size.
- QPP Predictor: Uses estimated gate counts and hardware specs (e.g., CPU info, network bandwidth) to estimate query runtime and \$ cost per EC2 instance.
- ILP Solver: Compares all EC2 options and selects the cost-effective instances deployment plan.
- S3 Storage Layer: Storage-Compute Disaggregation; Persists secret shares while enabling independent compute scaling.

Gate Count Model

- In MPC, AND gates require network rounds between parties, making total gate count the primary driver of execution time.
- Our Gate Count Model estimates gate per operator.
- Runtimes are proportional to the estimated gate counts.
- $R^2 = 0.99$



QPP Predictor

Predict secure query runtime and \$ cost for a given EC2 instance before execution.

- and_gate_benchmark -> query runtime
- > Optimal core count(OC) to run queries in parallel, maximizing the utilization of both CPU and network resources.
- Query runtime under different parallelism level.
- Forecast prediction to unseen queries and instances.

Input: query gate count and instance specs (e.g., CPU info, memory, network bandwidth).

Progress & Exploring:

- Linear regression for unseen query (single core), runtime accuracy > 85%.
- Identify optimal core count successfully. (gates/s -> network_usage/core)
- Actively exploring instance feature selection to improve accuracy (e.g., CPU clock speed + IPC, physical throughput of AES-NI/VAES), and other ML model based on both query&operator level.

ILP Solver

ILP Formulation

n Queries, m types of EC2 instances.

A. Constants

- E_i : q_i 's gates cnt
- C_j : I_j 's cost (\$/h)
- BW_j : I_j 's baseline network bandwidth
- $Core_j$: I_j 's vCPU number

B. Decision Variables

- $x_{i,j} \in \{0, 1\}$: 1 if query i is assigned to instance j , and 0 otherwise.
- $y_j \in \{0, 1\}$: 1 if instance j is activated, and 0 otherwise (used to account for the instance's fixed cost).

C. Objective Function: Minimize Total Cost

- $T_{i,j}$: the predicted runtime of q_i on I_j with w cores/queries in parallel
- $Duration_j = \max(T_{i,j}), k \in \{1, 2, \dots, w\}$

$$\min \sum_{j=1}^m (C_j \times Duration_j \times y_j \times 2)$$

D. Constraints

- Assignment constraint: each query must be assigned to exactly one pair of machine.
$$\sum_{j=1}^m x_{i,j} = 1, \quad \forall i$$
- Resource capacity constraint (parallelism): The total demand of queries running in parallel on a machine must not exceed its capacity.
$$\sum_i x_{i,j} \leq \min(OC_j, Core_j)$$

Goal:

Select a cost-efficient deployment to run a set of secure queries, balancing performance and cloud cost.

Approaches:

- Pareto Front Pruning: Filter instances that are dominated in both runtime and cost.
- Exploring Google OR-Tools to solve ILP problem.

Outcome:

Avoids trial-and-error deployments — execute directly on an optimal or near-optimal instance.